Lecture 26.07: Linear Regression with OLS (Normal Equation)

📌 Topic Overview

In this lecture, Shraddha ma'am presents the analytical closed-form solution to Linear Regression: **Ordinary Least Squares (OLS)** via the **Normal Equation**, which finds optimal parameters without any iterative loops or learning rates.

🏛️ The Normal Equation Formula

> $$\theta = (X_b^T X_b)^{-1} X_b^T y$$

Where:

- X_b is the feature matrix augmented with a column of ones for the bias: $X_b = [\mathbf{1}, X]$.
- $(X_b^T X_b)^{-1}$ is the matrix inverse.
- θ contains both the intercept $\theta_0 = b$ and weights $\theta_1, \ldots, \theta_d = w$.

💻 Python Implementation

```
import numpy as np

def linear_regression_ols_normal_equation(X, y):
    """
    Solves Linear Regression analytically using the Ordinary Least Squares (OLS)
    Normal Equation: theta = (X_b^T * X_b)^(-1) * X_b^T * y
    Zero iterations required; computes the exact global optimum in a single step.
    """
    m = len(y)
    # Add bias intercept column of ones to X: X_b = [1, X]
    X_b = np.c_[np.ones((m, 1)), X]

    # Analytical solution via Normal Equation
    theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)

    bias = theta_best[0]
```

```
    weights = theta_best[1:]

    return weights, bias

# --- Function Call & Testing ---
np.random.seed(42)
X_toy = 2 * np.random.rand(100, 1)
y_toy = 4 + 3 * X_toy.squeeze() + np.random.randn(100) * 0.1

w_ols, b_ols = linear_regression_ols_normal_equation(X_toy, y_toy)
print(f"True slope: 3.0 | OLS Weight: {w_ols[0]:.4f}")
print(f"True bias:  4.0 | OLS Bias:   {b_ols:.4f}")
```

🔑 Key Takeaways

1. OLS requires **zero iterations** and **no learning rate**.
2. It directly computes the exact mathematical global minimum.
3. However, inverting $(X^T X)$ has computational complexity $\mathcal{O}(d^3)$, which crashes when features $d > 10,000$.